

Tecnología Digital VI

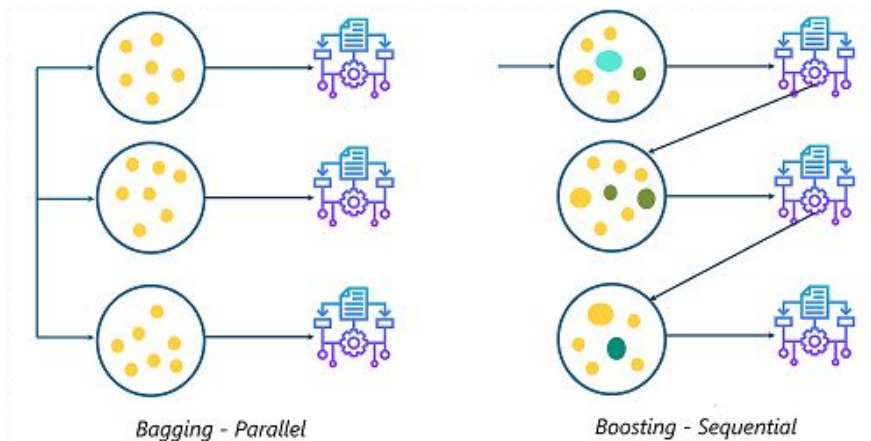
Ensamblés III – Boosting

Licenciatura en Tecnología Digital – UTDT

Boosting

A diferencia de los ensambles que vimos hasta el momento, en donde los modelos se combinan en paralelo, *Boosting* busca combinar diferentes modelos de manera secuencial para obtener una mejor predicción.

En general, en Boosting se van a utilizar *weak learners* (“tocones”), que se van mejorando sucesivamente.



Uno de los primeros y más clásicos algoritmos de boosting fue AdaBoost (Adaptive Boosting).

Lo distintivo de este algoritmo es que en cada modelo sucesivo del ensamble las muestras van tomando diferente peso en la métrica de optimización según qué tan bien predicha viniese cada muestra hasta el momento.

Si bien fue un algoritmo muy importante en el mundo de Machine Learning, actualmente suele ser superado en la práctica por otros algoritmos de boosting.

Boosting Básico

En la forma simple de Boosting, cada modelo trabaja sobre el árbol anterior.

En esta implementación aparecen algunos hiperparámetros:

- La cantidad de árboles, que ahora sí es una clara fuente de *overfitting*.
- El coeficiente de shrinkage lambda, controla la velocidad de aprendizaje.
- La cantidad de splits d o cualquier forma de caracterizar la complejidad de cada modelo individual (otra fuente de *overfitting*).

Algorithm 8.2 Boosting for Regression Trees

1. Set $\hat{f}(x) = 0$ and $r_i = y_i$ for all i in the training set.
2. For $b = 1, 2, \dots, B$, repeat:
 - (a) Fit a tree $\hat{f}^b$ with d splits ($d + 1$ terminal nodes) to the training data (X, r) .
 - (b) Update $\hat{f}$ by adding in a shrunk version of the new tree:

$$\hat{f}(x) \leftarrow \hat{f}(x) + \lambda \hat{f}^b(x). \quad (8.10)$$

- (c) Update the residuals,

$$r_i \leftarrow r_i - \lambda \hat{f}^b(x_i). \quad (8.11)$$

3. Output the boosted model,

$$\hat{f}(x) = \sum_{b=1}^B \lambda \hat{f}^b(x). \quad (8.12)$$

Gradient Boosting

En gradient boosting la idea es que cada nuevo modelo trabaje sobre el output del anterior utilizando el gradiente de la loss function como función de las predicciones.

Esto coloquialmente significa que cada nuevo modelo va a trabajar sobre los residuos de las predicciones previas.

1. Initialize $f_0(x) = \arg \min_{\gamma} \sum_{i=1}^N L(y_i, \gamma)$.

2. For $m = 1$ to M :

(a) For $i = 1, 2, \dots, N$ compute

$$r_{im} = - \left[\frac{\partial L(y_i, f(x_i))}{\partial f(x_i)} \right]_{f=f_{m-1}}.$$

(b) Fit a regression tree to the targets r_{im} giving terminal regions R_{jm} , $j = 1, 2, \dots, J_m$.

(c) For $j = 1, 2, \dots, J_m$ compute

$$\gamma_{jm} = \arg \min_{\gamma} \sum_{x_i \in R_{jm}} L(y_i, f_{m-1}(x_i) + \gamma).$$

(d) Update $f_m(x) = f_{m-1}(x) + \sum_{j=1}^{J_m} \gamma_{jm} I(x \in R_{jm})$.

3. Output $\hat{f}(x) = f_M(x)$.

Sabores de Gradient Boosting

Existen muchas implementaciones de gradient boosting que suman características determinantes para que estos algoritmos sean exitosos en la práctica.

Actualmente XGBoost, LightGBM, CatBoost son tres librerías muy utilizadas para tratar problemas complejos de predicción tabular.

Entender que XGBoost, LightGBM y CatBoost son tres tipos de Gradient Boosting

Tabular Data

Gradient-boosted decision trees (GBDTs) remain the go-to tabular competition winner's modelling tool, sometimes as part of an ensemble alongside neural nets.

Among winning solutions that used GBDTs, XGBoost and LightGBM were most popular this year, with 14 uses each. We found 8 winning solutions using CatBoost. Although part of the choice of library is likely due to familiarity, some winners explicitly stated that they experimented with all three of these libraries, and found significant performance differences between the different libraries on their dataset.¹⁵

AutoGluon was used for tabular prediction in two winning solutions. Most notably, the winner of Kaggle's Open Polymer Prediction 2025 competition used AutoGluon as part of an ensemble, and noted that AutoGluon "was able to beat an ensemble of XGBoost, LightGBM, and TabM models [tuned with] Optuna", while using only a fraction of the training budget.¹⁶

En el trabajo original de XGBoost se pueden encontrar explicaciones a las diferentes contribuciones de esta implementación particular de Gradient Boosting.

La primera contribución es la inclusión de la regularización dentro de la función objetivo.

2. TREE BOOSTING IN A NUTSHELL

We review gradient tree boosting algorithms in this section. The derivation follows from the same idea in existing literatures in gradient boosting. Specially the second order method is originated from Friedman et al. [12]. We make minor improvements in the regularized objective, which were found helpful in practice.

2.1 Regularized Learning Objective

For a given data set with n examples and m features $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}$ ($|\mathcal{D}| = n, \mathbf{x}_i \in \mathbb{R}^m, y_i \in \mathbb{R}$), a tree ensemble model (shown in Fig. 1) uses K additive functions to predict the output.

$$\hat{y}_i = \phi(\mathbf{x}_i) = \sum_{k=1}^K f_k(\mathbf{x}_i), \quad f_k \in \mathcal{F}, \quad (1)$$

where $\mathcal{F} = \{f(\mathbf{x}) = w_{q(\mathbf{x})}\} (q : \mathbb{R}^m \rightarrow T, w \in \mathbb{R}^T)$ is the space of regression trees (also known as CART). Here q rep-

Adicionalmente, agrega otras características para no overfitear. Una muy importante es que a los árboles sucesivos se les va ajustando el peso que tienen en la cuenta final para que cada vez aporten menos.

2.3 Shrinkage and Column Subsampling

Besides the regularized objective mentioned in Sec. 2.1, two additional techniques are used to further prevent overfitting. The first technique is shrinkage introduced by Friedman [11]. Shrinkage scales newly added weights by a factor η after each step of tree boosting. Similar to a learning rate in stochastic optimization, shrinkage reduces the influence of each individual tree and leaves space for future trees to improve the model. The second technique is column (feature) subsampling. This technique is used in RandomForest [4,

XGBoost también tiene como contribución varias formas diversas de calcular cómo los árboles subyacentes encuentran las decisiones para abrir nuevas ramas.

3. SPLIT FINDING ALGORITHMS

3.1 Basic Exact Greedy Algorithm

One of the key problems in tree learning is to find the best split as indicated by Eq (7). In order to do so, a split finding algorithm enumerates over all the possible splits on all the features. We call this the *exact greedy algorithm*.

3.2 Approximate Algorithm

The exact greedy algorithm is very powerful since it enumerates over all possible splitting points greedily. However, it is impossible to efficiently do so when the data does not fit entirely into memory. Same problem also arises in the dis-

3.3 Weighted Quantile Sketch

One important step in the approximate algorithm is to propose candidate split points. Usually percentiles of a fea-

3.4 Sparsity-aware Split Finding

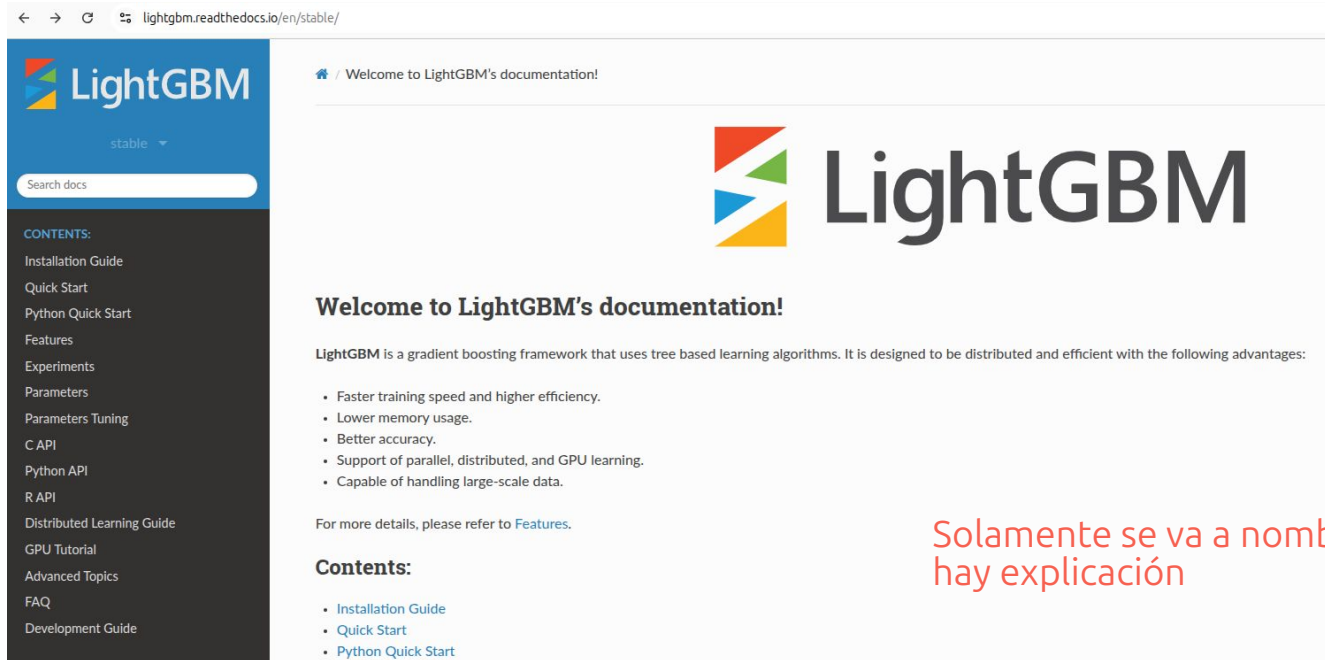
In many real-world problems, it is quite common for the input $\mathbf{x}$ to be sparse. There are multiple possible causes for sparsity: 1) presence of missing values in the data; 2) frequent zero entries in the statistics; and, 3) artifacts of feature engineering such as one-hot encoding. It is impor-

Al ser un algoritmo pensado para problemas complejos y reales, proporciona varias funcionalidades que están apuntadas a mejorar los tiempos de cómputo tanto en entrenamiento como en inferencia.

4. SYSTEM DESIGN

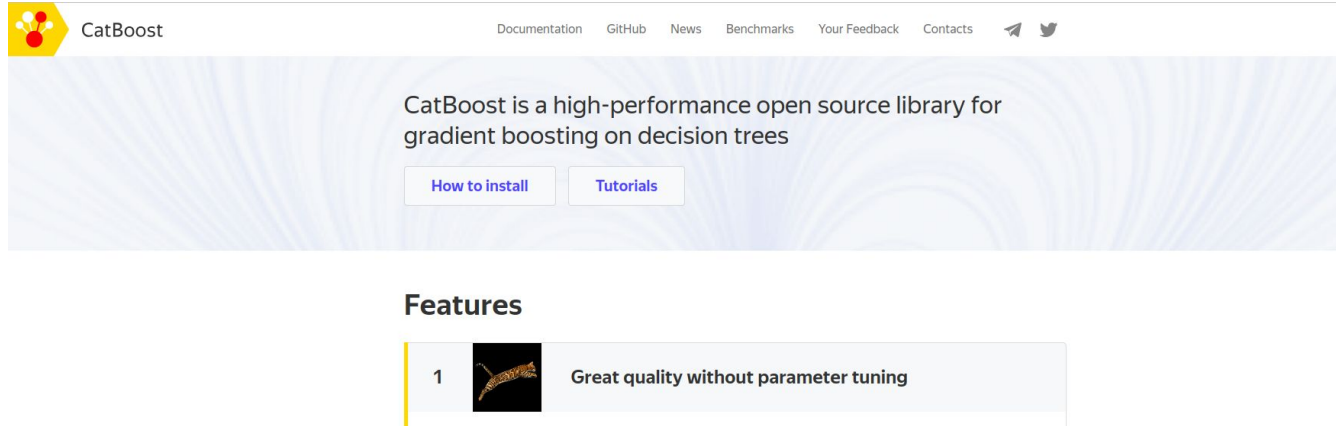
4.1 Column Block for Parallel Learning

The most time consuming part of tree learning is to get the data into sorted order. In order to reduce the cost of sorting, we propose to store the data in in-memory units, which we called *block*. Data in each block is stored in the compressed column (CSC) format, with each column sorted by the corresponding feature value. This input data layout only needs to be computed once before training, and can be reused in later iterations.



Solamente se va a nombrar, no hay explicación

LightGBM es una implementación muy competitiva propuesta por Microsoft. En su web (<https://lightgbm.readthedocs.io/en/stable/index.html>) se puede encontrar explicaciones de sus contribuciones así como de uso e ideas para manejar los hiperparámetros.



Lo mismo para este, no hay explicación

CatBoost es una implementación de GradientBoosting desarrollada por Yandex que tiene mucha adopción en la industria. En <https://catboost.ai/> se encuentra mucha información de uso, contribuciones, comparaciones, etc.

XGBoost vs LightGBM vs CatBoost

	XGBoost	LightGBM	CatBoost
Common Characteristics	1. Gradient-boosting decision-tree algorithm 2. based on decision trees 3. greedy thoughts		
Improvements & Benefits	1. parallel computing 2. second-order derivative of the loss function 3. linear model support; 4. regular term adding 5. shrinkage factor 6. column sampling 7. handling missing values	1. histogram algorithm 2. gradient-based one-side sampling (GOSS) 3. exclusive feature bundling (EFB) 4. leaf-wise algorithm 5. data parallel acceleration 6. cache optimizing	1. reducing hyper parameters tuning 2. interface for integration with scikit, as well as R and command-line interface 3. supporting categorical variables 4. self-defining loss functions 5. scalable GPU version 6. efficient deployment
Drawbacks	1. still need to traverse the dataset during the node splitting process 2. high space complexity of the pre-sorting process	1. may grow deeper decision trees in leaf-wise and produce over-fitting 2. sensitivity to noise	1. the processing of categorical features requires a lot of memory and time 2. random number settings influence the model prediction results
Hyper parameter	OC	learning_rate, max_depth, min_child_weight	learning_rate, max_depth, min_data_in_leaf
	CV	/	categorical_feature
	SC	colsample_bytree, subsample, n_estimators	feature_fraction, bagging_fraction, num_iterations

Where OC represents the hyperparameter used for overfitting, CV represents the hyperparameter used for categorical values, and SC represents the hyperparameter used for speed control.

Comparación de características de los 3 algoritmos. Adicionalmente se pueden encontrar comparaciones de benchmarks de todo tipo.

Optimización de hiperparámetros

Los modelos vistos son los ganadores en la práctica porque tienen varias mejoras implementativas más allá del algoritmo original teórico.

Adicionalmente, son modelos muy sensibles a sus *hiperparámetros* y la optimización de ellos es una parte sustancial de su éxito o fracaso.

Existen librerías que dan soporte para realizar la optimización de hiperparámetros mediante técnica como GridSearch, RandomSearch, etc.